CodeLens AI: An Integrated Educational Framework for Static Analysis, Runtime Visualization, Deterministic Code Repair, and Explainable AI Tutoring

Kala Chandrashekar
Department of Computer Science and Engineering
R V College of Engineering
Bengaluru, India
kalacl@rvce.edu.in

Rohith Arsha
Department of Computer Science and Engineering
R V College of Engineering
Bengaluru, India
rrohitarsha.scs25@rvce.edu.in

Dhruva K
Department of Computer Science and Engineering
R V College of Engineering
Bengaluru, India
dhruvak.scs25@rvce.edu.in

***Abstract*—Programming education often relies on compilers, interpreters, and Integrated Development Environments (IDEs) to identify programming errors. While these tools effectively detect syntax and semantic issues, they provide limited educational guidance for novice programmers regarding why errors occur, how algorithms execute, and how code quality can be improved. Recent Large Language Models (LLMs) offer natural language explanations but frequently rely on probabilistic reasoning, which may introduce inconsistent or hallucinated outputs when used directly for program analysis.**

This paper presents CodeLens AI, an integrated educational framework that combines deterministic static analysis, semantic program representation, runtime execution visualization, automated rule-based code repair, and local LLM-powered explanations into a unified programming assistance environment. The proposed framework first performs Abstract Syntax Tree (AST) generation, semantic extraction, rule-based bug detection, complexity estimation, and semantic graph construction. A secure runtime interpreter visualizes program execution without relying on Python's native `exec()` or `eval()` functions, while a deterministic Apply Fix engine provides rule-driven code corrections. Finally, a locally hosted Ollama-based language model generates educational explanations using the verified outputs of the deterministic analyzer rather than performing independent code analysis.

The framework emphasizes explainability, reliability, and educational value by separating deterministic program analysis from AI-assisted tutoring. Experimental validation demonstrates successful detection of common programming errors, runtime visualization of Python programs, automated code repair, and interactive AI-generated explanations within a unified educational environment.

***Index Terms*—Static Program Analysis, Abstract Syntax Tree (AST), Semantic Analysis, Runtime Visualization, Explainable Artificial Intelligence (XAI), Automated Code Repair, Programming Education, Python**

I. Introduction

Programming has become a fundamental skill across engineering, computer science, and data-driven disciplines. As the number of novice programmers continues to increase, educational tools capable of providing meaningful feedback during software development have become increasingly important. Traditional Integrated Development Environments (IDEs) and static analysis tools efficiently identify syntax errors, undefined variables, unused variables, and various code quality issues. However, these tools primarily target experienced developers and often provide concise diagnostic messages with limited educational context.

Recent advances in Large Language Models (LLMs) have enabled AI-assisted programming tools capable of generating explanations, debugging suggestions, and code completions. Despite these capabilities, purely LLM-driven systems frequently perform their own code analysis, which may produce inconsistent reasoning, incorrect diagnoses, or hallucinated explanations. Such behavior reduces their reliability in educational environments where correctness and explainability are essential.

To address these limitations, this work proposes **CodeLens AI**, an integrated educational programming framework that combines deterministic static program analysis with AI-assisted tutoring. Unlike existing AI coding assistants, the proposed framework separates deterministic analysis from language model reasoning. Static analysis, semantic extraction, complexity estimation, semantic graph generation, runtime interpretation, and automated code repair are performed using deterministic algorithms. The AI component receives only the

verified analysis results and generates educational explanations without independently identifying program errors.

The proposed architecture integrates multiple educational features within a single environment, including semantic visualization, algorithm complexity estimation, runtime execution tracing, deterministic code repair, and explainable AI tutoring. By combining these components into a unified workflow, CodeLens AI aims to improve both programming comprehension and debugging efficiency while maintaining reliable analysis results.

The primary contributions of this work are summarized as follows:

- Development of an integrated educational framework combining deterministic static analysis and AI-assisted tutoring.
- A semantic extraction pipeline that generates symbol tables, semantic graphs, and program representations for further analysis.
- A rule-based static analysis engine capable of detecting common programming errors and code quality issues.
- A secure runtime interpreter that visualizes program execution without executing arbitrary Python code.
- A deterministic Apply Fix engine that performs safe rule-based code corrections.
- A local Ollama-powered explanation layer that generates educational feedback using verified analysis results instead of performing independent code analysis.

II. System Architecture

The proposed CodeLens AI framework follows a modular layered architecture in which each component performs a specific responsibility within the program analysis pipeline. Figure **??** illustrates the overall system architecture.

The framework begins by parsing Python source code into an Abstract Syntax Tree (AST), which forms the foundation for all subsequent analysis stages. The Semantic Extraction module traverses the AST to construct symbol tables, identify variables, functions, scopes, loops, conditional statements, function calls, and semantic relationships between program entities.

The extracted semantic representation is then consumed by multiple deterministic analysis modules. The Rule Engine identifies common programming mistakes such as undefined variables, infinite loop risks, missing recursion base cases, binary search logic errors, dead code, unreachable code, and syntax errors. Simultaneously, the Complexity Engine estimates algorithmic characteristics including time complexity, space complexity, cyclomatic complexity, maintainability score, function count, and loop count using heuristic-based static analysis.

A Semantic Graph Builder constructs an interactive graph representation of program entities and their relationships, allowing users to visualize dependencies between functions, variables, loops, and control-flow constructs.

To improve program comprehension, the Runtime Engine safely interprets supported Python programs without invoking Python's native execution mechanisms such as `exec()` or `eval()`. Instead, the interpreter executes supported Abstract Syntax Tree nodes while maintaining runtime state, execution timelines, variable updates, and console output.

When supported deterministic findings are detected, the Apply Fix Engine performs rule-based source code modifications. Unlike AI-generated code editing, these fixes are deterministic, reproducible, and restricted to predefined transformation rules.

Finally, the AI Explanation Layer communicates with a locally hosted Ollama language model. Rather than analyzing source code independently, the language model receives verified findings, complexity metrics, runtime summaries, and relevant source snippets generated by the deterministic analyzer. This architectural separation ensures that AI-generated explanations remain consistent with verified program analysis while minimizing hallucinated outputs.

III. Methodology

The proposed framework processes Python programs through a deterministic multi-stage analysis pipeline before generating educational explanations. Figure 1 illustrates the overall workflow.

Initially, the source program is parsed into an Abstract Syntax Tree (AST), which serves as the structural representation of the program. The Semantic Extractor traverses the AST to identify functions, variables, scopes, assignments, loops, conditional statements, imports, recursion candidates, and control-flow events. These semantic entities are stored in an intermediate semantic representation that supports subsequent analysis modules.

The Rule Engine evaluates the semantic representation using deterministic rule sets designed to detect common programming mistakes. Examples include undefined variables, infinite loop risks, missing recursion base cases, binary search implementation errors, dead code, unreachable branches, and syntax errors. Since these rules operate directly on semantic information rather than probabilistic reasoning, identical source programs consistently produce identical findings.

Following rule evaluation, the Complexity Engine estimates algorithmic properties of the program. The engine computes heuristic approximations of time complexity, space complexity, cyclomatic complexity, maintainability score, function count, loop count, and source code statistics using structural information extracted from the AST.

The Runtime Engine then interprets supported Python constructs using a secure AST interpreter. Execution state, variable updates, runtime steps, and console output are recorded to provide an interactive execution timeline without executing arbitrary Python code.

When deterministic findings correspond to supported correction patterns, the Apply Fix Engine performs predefined source code transformations. These transformations include deterministic fixes for undefined variables, missing recursion base cases, binary search boundary updates, infinite loop risks, and unreachable code.

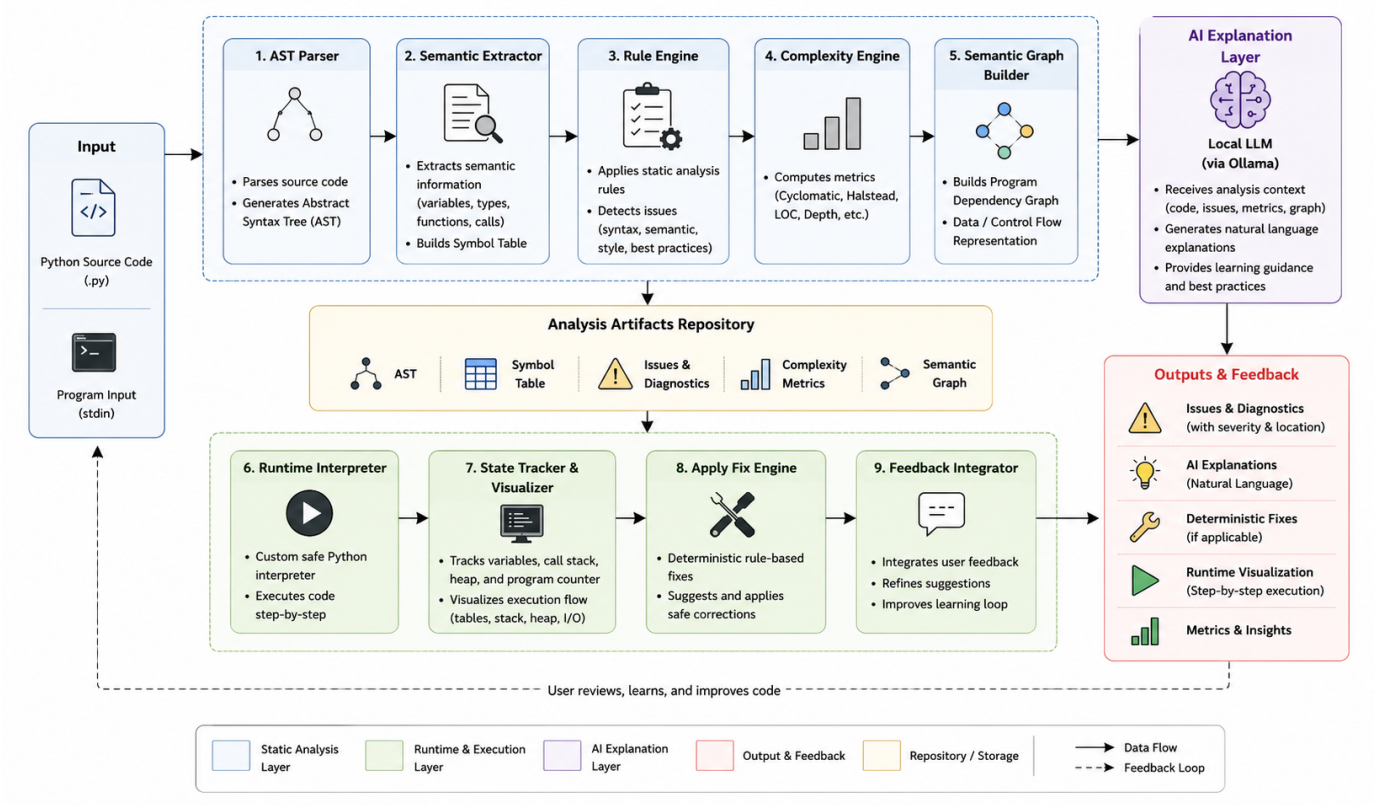

Fig. 1. Overall methodology of the proposed CodeLens AI framework. The system transforms Python source code into an Abstract Syntax Tree (AST), performs semantic extraction, rule-based analysis, complexity estimation, runtime interpretation, deterministic code repair, and finally generates AI-assisted educational explanations using a local Large Language Model (LLM).

Finally, the AI Explanation Layer constructs a focused prompt using the verified outputs of the deterministic analyzer. The prompt includes relevant source code fragments, identified findings, complexity metrics, and optional runtime summaries. A locally hosted Ollama language model then generates educational explanations, corrected examples, learning tips, and best practices while relying entirely on deterministic analysis results instead of independently analyzing the source code.

IV. Results

The current implementation of CodeLens AI successfully integrates deterministic static analysis, runtime visualization, automated code repair, semantic graph generation, and AI-assisted educational explanations within a unified programming environment.

The static analysis engine accurately detects several categories of programming issues, including undefined variables, infinite loop risks, missing recursion base cases, binary search logic errors, dead code, unreachable code, and syntax errors. Interactive semantic graphs allow users to visualize relationships among program entities including variables, functions, loops, and control-flow structures.

The proposed runtime interpreter executes supported Python programs within a secure execution environment, providing execution timelines, variable state visualization, console output, and structured runtime errors without invoking Python's native execution mechanisms. The deterministic Apply Fix engine automatically repairs supported programming mistakes using predefined transformation rules while preserving reproducibility.

The AI Explanation Layer integrates a locally hosted Ollama language model to provide educational explanations derived from verified deterministic analysis results. This architecture ensures that AI-generated feedback remains consistent with the findings produced by the underlying analysis framework while avoiding the need for probabilistic program analysis.

At the time of writing, the framework has successfully passed more than 100 automated backend regression tests together with frontend type checking, linting, and production build validation. These tests verify the correctness of the implemented analysis modules, runtime interpreter, deterministic code repair engine, and AI explanation workflow. Comprehensive experimental evaluation, user studies, and comparisons with existing educational programming tools will be presented in the final version of this work.

References

[1] Z. Fan, X. Gao, M. Mirchev, A. Roychoudhury, and S. H. Tan, "Automated Repair of Programs from Large Language Models," *arXiv preprint arXiv:2205.10583*, 2022.

[2] S. Kang, B. Chen, S. Yoo, and J.-G. Lou, “Explainable Automated Debugging via Large Language Model-Driven Scientific Debugging,” *arXiv preprint arXiv:2304.02195*, 2023.

[3] Y. Liu, A. K. Singh, and A. Chandra, “Program Repair Guided by Datalog-Defined Static Analysis,” In *Proceedings of the ACM SIGSOFT International Symposium on the Foundations of Software Engineering (FSE)*, 2023.

[4] S. Dikici and T. T. Bilgin, “Advancements in Automated Program Repair: A Comprehensive Review,” *Knowledge and Information Systems*, Springer, vol. 67, pp. 4737–4783, 2025.

[5] I. Malashin, “Generation of Natural-Language Explanations for Static-Analysis Warnings Using Single- and Multi-Objective Optimization,” *Computers*, vol. 14, no. 12, 2025.

[6] J. Sun, Q. V. Liao, M. Muller, M. Agarwal, S. Houde, K. Talamadupula, and J. D. Weisz, “Investigating Explainability of Generative AI for Code through Scenario-Based Design,” *arXiv preprint arXiv:2202.04903*, 2022.

[7] H. Ding *et al.*, “A Static Evaluation of Code Completion by Large Language Models,” *arXiv preprint arXiv:2306.03203*, 2023.

[8] Y. Liu *et al.*, “Refining ChatGPT-Generated Code: Characterizing and Mitigating Code Quality Issues,” *arXiv preprint arXiv:2307.12596*, 2023.

[9] S. Stock *et al.*, “Application of Artificial Intelligence to Formal Methods: An Analysis of Current Research,” *Empirical Software Engineering*, Springer, 2025.

[10] A. Faraji *et al.*, “AI-Driven Software Test Automation: A Systematic Review,” *IEEE Access*, 2025.